Skill registry — 2026-05-21

A list of every custom skill I've written for Claude Code (Anthropic's coding assistant CLI), with measured token usage where I have receipts and labeled estimates where I do not. A "skill" here is a reusable slash command — a named recipe Claude Code runs when I type `/audit` or similar. 4 repos, 33 skills, 12 of them with both a guess and a measurement so far.

MEASURED TOKEN COST — LAST 90 DAYS

4.36M ~30% of /review

All custom skills (measured across this portfolio)

14.4M 14 runs

/review built-in (Claude Code's own slash command, shown for scale)

My entire custom portfolio cost about 30% of what /review alone cost in the same window. /review is excluded from every total below. Same bar scale — no zoom trickery.

Most rows here are guesses. I wrote a skill, imagined someone using it 30 times a year, wrote that down. The rows tagged ✓MEASURED are the ones where I went back to my Claude Code transcripts and counted — those are facts. The rest are tagged ○ ESTIMATE and are guesses by the person who built the skill (that's me). In every measured case so far, my original guess was off — usually by 5× to 100×. The interesting part isn't that I was wrong. The interesting part is that you can see exactly how wrong, on page 2.

Per-repo summary

REPO	SKILLS	MEASURED	TOKENS USED (90D)	SAVED / YR*
AudiobookMaker	8	2	202K	~1.99M
Spacepotatis	12	4	930K	~10.1M
claude-skills	9	4	2.67M	~23.9M
mikkonumminen.dev	4	2	561K	~4.99M

** Saved is a model, not a measurement — see the method page. Italicised values are majority-estimate; upright values are majority-measured.*

Calibration honesty — where my guesses landed

Here are the rows where I had a guess before I had data. The “How wrong” column is the measurement divided by the guess. Green means I landed within ±10%. Orange means I was off by 5× or more in either direction. 5 of 12 rows are orange. The fix is not to write better guesses next time. The fix is to keep measuring.

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
AUDIOBOOKMAKER **release-cut**	2K	188K	**111×** under
SPACEPOTATIS **equipment**	4K	401K	**93×** under
CLAUDE-SKILLS **mikko-help**	7K	644K	**92×** under
SPACEPOTATIS **new-story**	5K	56K	**10×** under
CLAUDE-SKILLS **skill-usage**	4K	24K	**6.1×** under
AUDIOBOOKMAKER **copyright-scan**	3K	14K	**4.5×** under
SPACEPOTATIS **content-audit**	15K	53K	**3.5×** under
CLAUDE-SKILLS **ai-codegen-smell-audit**	75K	199K	**2.7×** under
SPACEPOTATIS **save-roundtrip-audit**	12K	19K	**1.5×** under
CLAUDE-SKILLS **audit**	425K	553K	**1.3×** under
MIKKONUMMINEN.DEV **sync-readmes**	141K	119K	**1.2×** over

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
MIKKONUMMINEN.DEV **skill-registry**	130K	148K	**1.1×** under

Per-repo skill tables

One row per skill. Measured rows have a green wash; estimated rows are white. **Saved / yr** is a model — italic numbers mean it's modeled on top of an estimate, upright numbers mean it's modeled on top of a measurement. The model itself is described on the method page.

AudiobookMaker

8 skills · 2 measured · github.com/MikkoNumminen/AudiobookMaker

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
ci-failure-triage When CI fails on a release tag or PR, walk the failure modes in the right order against a recipe library built from the project's fix(ci) commit history.	○ ESTIMATE	**2K** tokens / use (est.)	89 / yr (est.)	*n/a*	*~338K/yr* modeled (from est.)
copyright-scan Scan a git diff (staged by default, or a named revision range / file set) for accidental third-party copyright leaks before they land on origin.	✓MEASURED last 2026-05-12	**14K** tokens / use (measured)	1 in 90d ~4/yr projected	**4.5× under** guess was 3K/use	~111K/yr modeled
pronunciation-corpus-add Append a Finnish pronunciation failure (a word the Chatterbox Grandmom voice mispronounces) to the project's pronunciation corpus at docs…	○ ESTIMATE	**2K** tokens / use (est.)	—	*n/a*	—
release-bundle-audit Audit the AudiobookMaker PyInstaller release bundle for unused dependencies, dead-code data files, and accidental ML-stack pollution; then propose spec-only fixes on a `chore/release-bundle-size` branch.	○ ESTIMATE	**4K** tokens / use (est.)	4 / yr (est.)	*n/a*	*~30K/yr* modeled (from est.)
release-cut Cut a new AudiobookMaker release (bump APP_VERSION, tag vX.Y.Z, push, verify CI lands SHA-256 in release notes AND sidecar). Use this ski…	✓MEASURED last 2026-05-12	**188K** tokens / use (measured)	1 in 90d ~4/yr projected	**111× under** guess was 2K/use	~1.51M/yr modeled

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
voice-pack-finnish Build a Finnish voice pack from a source audio file end-to-end — chunked analyze with ECAPA diarization, transcript validation per speaker, branching to LoRA training (long sources) or few-shot ref-clip packaging (short sources), and ear-check by synth.	○ ESTIMATE	**6K** tokens / use (est.)	—	*n/a*	—
work-session Start, pause, or finish a TODO.md work session as one of the 4 permanent Claude sessions in AudiobookMaker. Use this whenever the user sa…	○ ESTIMATE	**2K** tokens / use (est.)	—	*n/a*	—
worktree-launch Start a new parallel Claude session safely.	○ ESTIMATE	**1K** tokens / use (est.)	—	*n/a*	—

Spacepotatis

12 skills · 4 measured · github.com/MikkoNumminen/Spacepotatis

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
balance-review Diff uncommitted changes to game data (weapons, enemies, waves, missions, perks, augments, loot pools, solar systems) and report DPS, TTK, energy-cost-per-DPS, augment-folded effective DPS, loot-pool roster shifts, and drop-rate deltas vs HEAD.	○ ESTIMATE	**14K** tokens / use (est.)	50 / yr (est.)	*n/a*	*~1.35M/yr* modeled (from est.)
content-audit Pre-commit content invariants check — orphan refs (enemy / weapon / sprite / pod / loot-pool / mission system / story trigger), missing sprite generators, perk drop-weight sanity, mission prereq DAG, story integrity (voice + music files, trigger refs), storyTriggers helper coverage.	✓ MEASURED last 2026-05-03	**53K** tokens / use (measured)	1 in 90d ~4/yr projected	**3.5× under** guess was 15K/use	~425K/yr modeled
equipment Create, modify, or remove a weapon or piece of equipment (augment / reactor / shield / armor) — including visual changes to how it appears in combat or in the loadout UI.	✓ MEASURED last 2026-05-03	**401K** tokens / use (measured)	2 in 90d ~8/yr projected	**93× under** guess was 4K/use	~6.42M/yr modeled

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
modular-architecture-audit Orchestrates the multi-phase modular-architecture audit + refactor.	○ ESTIMATE	**5K** tokens / use (est.)	5 / yr (est.)	*n/a*	*~50K/yr* modeled (from est.)
new-enemy Scaffold a new enemy — enemies.json entry, BootScene placeholder sprite generator, optional test wave, and integrity test verification.	○ ESTIMATE	**6K** tokens / use (est.)	25 / yr (est.)	*n/a*	*~275K/yr* modeled (from est.)
new-migration Add a Postgres schema migration end-to-end — dated SQL file in db/migrations/, Database interface update in src/lib/db.ts, prod applicati…	○ ESTIMATE	**7K** tokens / use (est.)	15 / yr (est.)	*n/a*	*~210K/yr* modeled (from est.)
new-mission Scaffold a new combat mission across missions.json, waves.json, the galaxy planet binding, and a smoke test — in one shot.	○ ESTIMATE	**8K** tokens / use (est.)	30 / yr (est.)	*n/a*	*~480K/yr* modeled (from est.)
new-perk Scaffold a new mission-only perk — perks.ts entry, BootScene icon generator, HUD chip wiring, and (for actives) a switch case in PerkCont…	○ ESTIMATE	**9K** tokens / use (est.)	10 / yr (est.)	*n/a*	*~180K/yr* modeled (from est.)
new-solar-system Add a new solar system to the galaxy — solarSystems.json entry (incl. galaxy bed), SolarSystemId union extension, optional unlock gating,…	○ ESTIMATE	**13K** tokens / use (est.)	5 / yr (est.)	*n/a*	*~130K/yr* modeled (from est.)
new-story Add, modify, or remove story content — cinematic popups, voiceovers, music beds, body text, written synopses, and auto-trigger wiring.	✓ MEASURED last 2026-04-30	**56K** tokens / use (measured)	1 in 90d ~4/yr projected	**10× under** guess was 5K/use	~444K/yr modeled
new-weapon *Superseded by /equipment.*	*REDIRECT*	—	—	*n/a*	—
save-roundtrip-audit Pre-commit save-pipeline invariants check — walks every StateSnapshot field through the 8 layers of the save round-trip (snapshot interfa…	✓ MEASURED last 2026-05-03	**19K** tokens / use (measured)	1 in 90d ~4/yr projected	**1.5× under** guess was 12K/use	~149K/yr modeled

claude-skills

9 skills · 4 measured · github.com/MikkoNumminen/claude-skills

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
ai-codegen-smell-audit Read-only audit for specific failure modes that recur in LLM-generated code — defensive guards on impossible cases, generic names in domain code, swallowed errors, single-use helpers, mirror tests, and so on.	✓MEASURED last 2026-05-17	**199K** tokens / use (measured)	1 in 90d ~4/yr projected	**2.7× under** guess was 75K/use	~1.60M/yr modeled
audit Run a multi-phase robustness audit of the current codebase.	✓MEASURED last 2026-05-20	**553K** tokens / use (measured)	2 in 90d ~8/yr projected	**1.3× under** guess was 425K/use	~8.84M/yr modeled
mikko-audit-suite Run every `mikko-*` audit skill that's relevant to the current codebase, in a sensible order, and produce one index document linking to each audit's report.	○ ESTIMATE	—	18 / yr (est.)	*n/a*	—
mikko-help List every installed `mikko-*` skill with its description (or with its `barney:` field when `--barney` is passed).	✓MEASURED last 2026-05-20	**644K** tokens / use (measured)	2 in 90d ~8/yr projected	**92× under** guess was 7K/use	~10.3M/yr modeled
mikko-install Install, update, list, or uninstall `mikko-*` skills from the cloned `claude-skills` source repo into the user-wide skill directory (`~/....	○ ESTIMATE	**4K** tokens / use (est.)	—	*n/a*	—
react-anti-patterns-audit Read-only audit for six concrete React anti-patterns that recur in component code (key=index in lists, useEffect for derived state, dep-array lies, state mutation instead of replace, missing effect cleanup, multiple sources of truth).	○ ESTIMATE	**23K** tokens / use (est.)	30 / yr (est.)	*n/a*	*~1.35M/yr* modeled (from est.)
readme-drift-sync Detect drift between what the repo actually contains and what `README.md` claims, then rewrite ONLY the drifted sections in the README's ...	○ ESTIMATE	**40K** tokens / use (est.)	15 / yr (est.)	*n/a*	*~1.20M/yr* modeled (from est.)
security-audit Orchestrates the multi-phase security audit + remediation.	○ ESTIMATE	—	—	*n/a*	—

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
skill-usage Measure actual Claude Code skill usage from local transcript JSONL files.	✓MEASURED last 2026-05-21	**24K** tokens / use (measured)	3 in 90d ~12/yr projected	**6.1× under** guess was 4K/use	~584K/yr modeled

mikkonumminen.dev

4 skills · 2 measured · github.com/MikkoNumminen/mikkonumminen.dev

SKILL	STATUS	COST / USE	RUNS	CALIBRATION	SAVED / YR
md-to-pdf Render a markdown report (or any HTML) to a styled PDF using the local Chrome's --print-to-pdf flag.	○ ESTIMATE	**25K** tokens / use (est.)	10 / yr (est.)	*n/a*	*~500K/yr* modeled (from est.)
skill-localUpdate Refresh every local artifact the site renders about your portfolio skills — the JSON the /contact terminal fetches at runtime, the measurement-overlaid annual totals, and the downloadable skills-registry PDF — in one sequenced chain.	○ ESTIMATE	**83K** tokens / use (est.)	—	*n/a*	—
skill-registry Scan every sibling repo under D:/koodaamista for `.claude/skills/*/SKILL.md` files and emit a consolidated registry as a structured JSON …	✓MEASURED last 2026-05-20	**148K** tokens / use (measured)	3 in 90d ~12/yr projected	**1.1× under** guess was 130K/use	~3.54M/yr modeled
sync-readmes Audit project data against sibling repos' READMEs and open a PR with drift corrections.	✓MEASURED last 2026-05-18	**119K** tokens / use (measured)	1 in 90d ~4/yr projected	**1.2× over** guess was 141K/use	~949K/yr modeled

Method, in plain language

How the numbers in this document are produced, what they mean, and where I am still guessing.

How “measured” rows are produced

Every time I run a Claude Code session (one conversation in the CLI from start to exit), the harness — the runtime that drives the CLI between me and the model — writes a transcript to `~/.claude/projects/<dir>/<sessionId>.jsonl`. The file is JSON Lines: one JSON object per line, one line per message. Each assistant message in those files carries an `attributionSkill` field when a skill is active. That field is the trail.

A separate skill called `skill-usage` walks every transcript file, filters to assistant messages with an `attributionSkill`, groups by (sessionId, skillName), sums the input + output + cache-creation tokens, and dedupes by `requestId` so retried API calls do not double-count. The output is a dated `SKILL-USAGE-{YYYY-MM-DD}.json` under `.claude/agent-verdicts/`. That JSON is what the renderer reads to populate every “measured” row in this document.

Window: the last 90 days from the moment `skill-usage` ran. Sessions older than 90 days are ignored even if they're still on disk. The boundary is inclusive at the start (a session exactly 90 days old, to the second, is counted) and exclusive at the end (the very moment the scanner runs is the cutoff). Running the scanner an hour later can therefore drop a session that just crossed the boundary — that's a real if rare flake, and re-running the chain re-derives the right answer.

Annual projection: dumb linear math. If a skill ran twice in 90 days, the annual figure says about 8. That is on purpose. I would rather show the projection method honestly than dress up a single data point as a trend. When a row says *“1 in 90d → ~4/yr projected”* you should read that as *“barely a data point — trust the cost-per-use, ignore the annual.”*

Cache accounting: the per-invocation total sums `input_tokens`, `output_tokens`, and `cache_creation_input_tokens`. It does *not* sum `cache_read_input_tokens` — those are cache hits paid for upstream and roughly 10× cheaper. Counting them would double-bill a single skill's multi-turn run. If you care specifically about cache efficiency, that's a different report.

What is not counted: any skill that did not run in the 90-day window has no measured row, even if I use it often outside that window. Sub-agent token costs (work the parent skill delegates to a parallel Claude Code agent,

written to its own transcript file under a `subagents/` sibling directory) are included where the harness logs them — each sub-agent transcript inherits the parent session's `attributionSkill`. If a skill spawns work the harness does not tag, those tokens are invisible to this measurement.

Renamed skills: there's a small in-source rename map (`RENAMED_SKILLS` in `apply-measurement-overlay.mjs`) that retargets historical old-name measurements onto the renamed registry row. Without this, a rename would silently delete ~90 days of measured signal from the rendered totals. Entries get retired once the window rolls past the rename date.

How “estimated” rows are produced

> I made up the number. I imagined someone — usually me — running the skill some number of times a year and costing some number of tokens per use, and I wrote down what felt right. There is no math behind these. They are guesses by the person who built the skill, written down before any measurement existed.

The estimates are still in the document because the alternative is showing only a third of the portfolio. The rows that have never been invoked in the 90-day window still represent real tools that take real tokens when used. Pretending they don't exist would make the picture cleaner and less true.

Make the asymmetry visible: every measured estimate I had has turned out to be wrong, usually low by 5× to 100×. Apply the same skepticism to the rows that have not been measured yet. If anything, the un-measured estimates are likely to be *more* wrong, because the ones I measured first were the ones I felt most confident about.

How “tokens saved” is computed

The savings number is a model, not a measurement. Here is the model in one line:

Saved = (cost of doing it the unstructured way − cost of doing it with the skill) × annual uses. I model the unstructured way as ~3× a focused skill run, because an unstructured chat would scout the files, talk through a plan, pick an approach, and only then write the same code. So saved ≈ 2× cost-per-use × annual uses.

The 3× number comes from a handful of side-by-side runs I did on my own machine. It is not a benchmark. It is not a guarantee. It is the number I'm using until I have more measured baselines, and it is the single load-bearing assumption in every "Saved / yr" cell in this document. Per-skill overrides exist (`tokens_saved_per_use`

on a receipt) for cases where the heuristic is obviously wrong — a redirect skill, for example, has no replacement work to compare against. Few skills currently override it.

Cost appears on both sides of that subtraction, so the savings figure is more sensitive to bad guesses than the cost figure is. An estimated row with a 100× under-guess on cost-per-use produces a 100× under-guess on savings, too. The italicised numbers in the "Saved / yr" column are exactly those rows: a model stacked on top of a guess. Treat them as a lower bound at best.

What this document does NOT claim

- No production cost numbers. This is development-time tooling on my own machine.
- No comparison to other engineers' setups. I only have transcripts for one engineer.
- No per-feature ROI claim. This is per-skill cost, not per-feature value.
- No assertion that any of this scales linearly to a team. It might. I have not measured.
- No claim that the modeled savings would survive an actual A/B test against well-prompted unstructured chats. They might; I haven't tested.

Reproducibility

From inside this repo (`mikkonumminen.dev/`) on a machine with Claude Code installed:

1. `/mikko-skill-usage` — writes `.claude/agent-verdicts/SKILL-USAGE-LATEST.json` from local transcripts.
2. `/skill-localUpdate` (formerly `/skill-pdf`) — re-walks the portfolio inventory, applies the measurement overlay, renders this PDF.

The data the renderer reads is committed to the repo at `public/data/skills-registry.json` . If you want to inspect what's behind a number on this page, that file is the source of truth. The dated history lives in `.claude/agent-verdicts/SKILL-REGISTRY-*.json` .

The *last <date>* marker on a measured row is the timestamp of the most recent Claude Code transcript that invoked that skill. The transcript itself stays on my machine because it contains code and context from private repos. If you want to verify the measurement methodology rather than the measurement, the parser source is at `~/.claude/skills/mikko-skill-usage/SKILL.md` — it is a JSONL parser, not a network call to anything.

Generated 2026-05-21 at 02:09 UTC. Custom-skill measured cost in the 90-day window: 4.36M tokens. Modeled savings 85% measurement-backed, 15% estimate-backed. Built-in /review reference excluded from every total above.